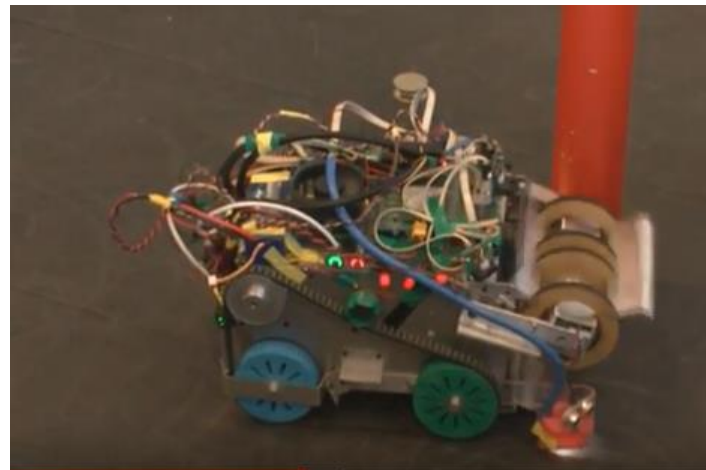


ENMT301

Mechatronics System Design:

Lecture & Lab : Software Project Management



Richard Clare

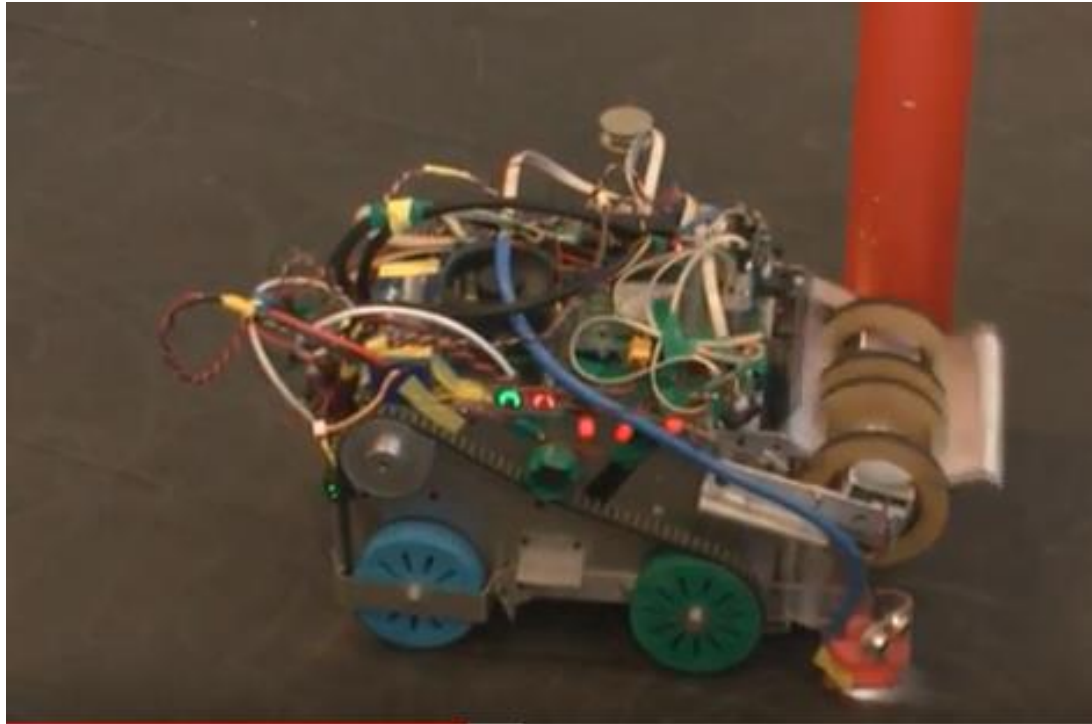
richard.clare@canterbury.ac.nz

Overview for Today

- Some software fundamentals
- Arduino code & Sketch tool
- Task management for the Robocup project
- Example of how to structure your code
- Version Control with Git (interactive)

RoboCup

- You have many sensors to detect distances and colour
- You have motors (DC, stepper) to control



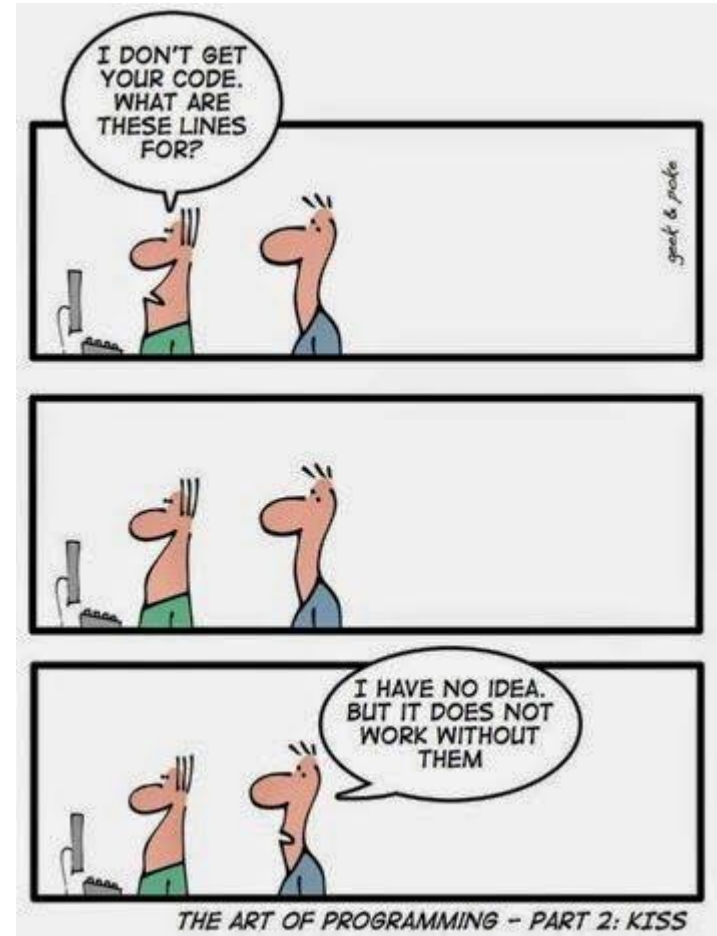
RoboCup 2

- You need to navigate
- You need to write software with your Arduino microcontroller to do all this



Comment your code

- Write Programs for People, not Computers!
- Your team-mate doesn't know what you are thinking!
- Your future self might not know what you are thinking!
- Example:



```
#define LEFT_MOTOR_ADDRESS 1 //Pin corresponding to the left dc motor
```

Naming conventions

- Make names consistent, distinctive and meaningful
- For example:
temperature
- Is better than:
t, temp, x, val, value, answer, bob, fred ...

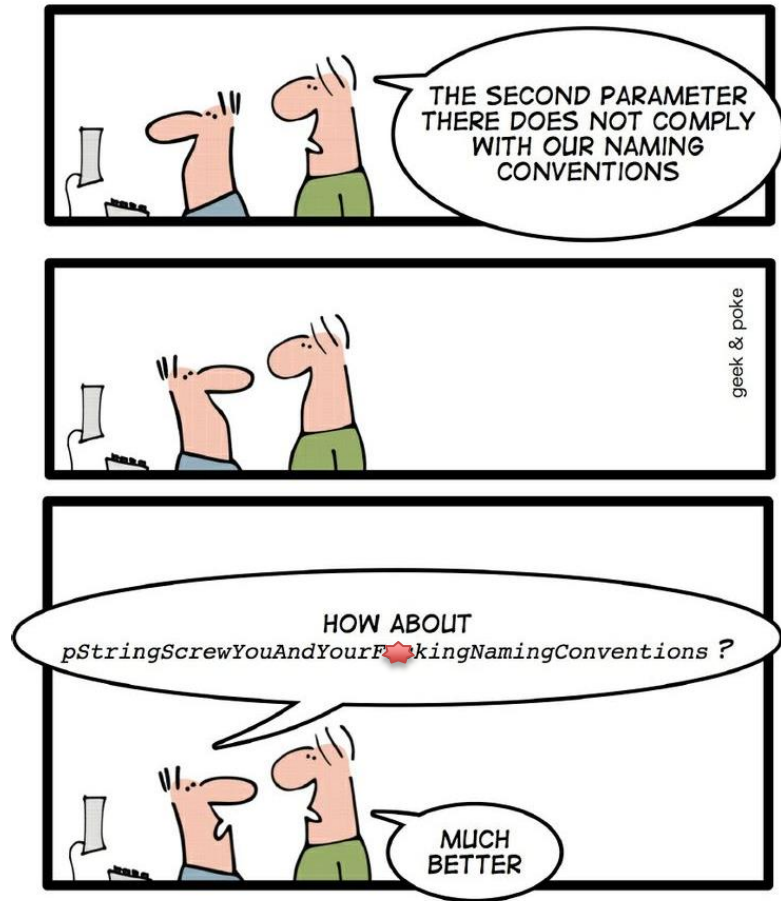
Naming conventions 2

- Do not mix conventions like this. Your team will end up in a shambles

```
int firstMotorSpeed  
int SecondMotorSpeed  
int third_motor_speed  
int Fourth_Motor_Speed
```

Naming conventions 3

- Which convention doesn't matter – having a convention does!
- Adopt a variable naming convention in your team and stick with it
- eg `set_motor` (underscore between words)
- or `setMotor` (camel Case)



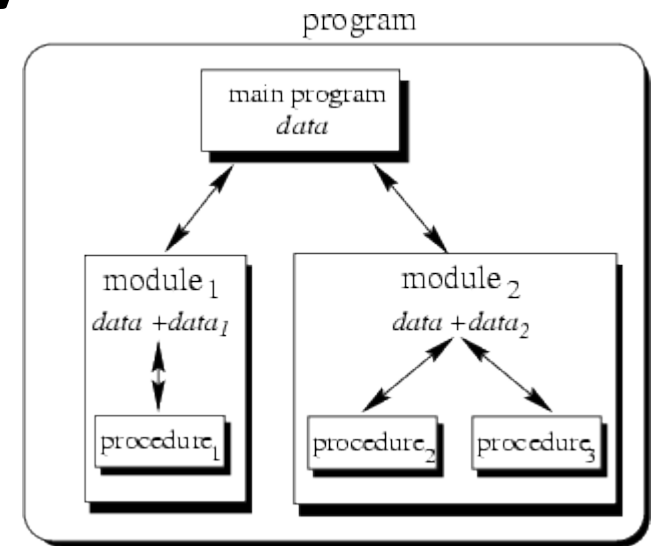
Don't embed constants

- Define constants exactly once in project (what if you want to change it?)
- Don't define constants within functions
- Good to define at start of main (.ino) file all in same place
- Or in header file relating to that component
- Example:

```
#define US_READ_TASK_PERIOD 40
```

Modularity

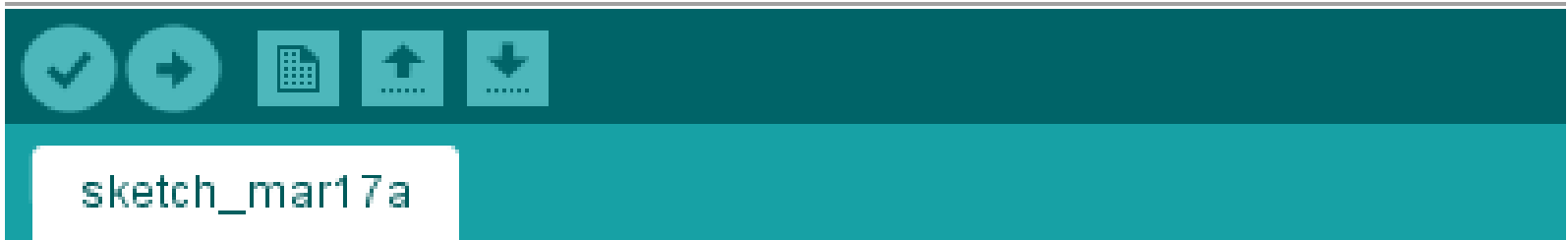
- Divide into subprograms
not all code in 1 main() or file!
- Easier to understand
- Don't repeat yourself
- Anything repeated in two or more places will eventually be wrong in at least one
- Cuts the amount of testing required



Arduino code structure

sketch_mar17a | Arduino 1.8.1

File Edit Sketch Tools Help



```
void setup() {  
    // put your setup code here, to run once:  
  
}  
  
void loop() {  
    // put your main code here, to run repeatedly:  
  
}
```

Template for Robocup

- We have developed a template from previous projects for you to use and modify if you wish. Optional!
- Modularized
- It has some structure but empty of functionality
- The skeleton code does compile on the Arduino
- robocup_template is available on ENMT301 Learn page
- Use this as your initial commit for your git repository (we will discuss later)!

Arduino C++ code

- The code for Arduino is C++
- There are header files .h
- There are .cpp files
- There is a main .ino file
(robocup_template.ino)

.h files

- Function prototypes & relevant constants

robocup_template - motors.h | Arduino 1.8.1

File Edit Sketch Tools Help

```

//*****
//      motors.h
//*****

#ifndef MOTORS_H_
#define MOTORS_H_

// SET THIS TO REAL VALUES
#define LEFT_MOTOR_ADDRESS 1      //Pin corresponding to the left dc motor
#define RIGHT_MOTOR_ADDRESS 1    //Pin corresponding to the right dc motor
#define MIN_SPEED_CAP 1         //Set the minimum speed value that can be written to the motors
#define MAX_SPEED_CAP 1         //Set the maximum speed value that can be written to the motors

void check_speed_limits(/*parameters*/);
void set_motor(/*parameters*/);

#endif /* MOTORS_H_ */
```

.cpp files

- Relevant functions

```
#include "motors.h"
#include "Arduino.h"

/* Check whether the speed value to be written is within the maximum
 * and minimum speed caps. Act accordingly.
 *
 */
void check_speed_limits(/*parameters*/) {
    Serial.println("Check the motor speed limit \n");
}

/* In this section, the motor speeds should be updated/written.
 *It is also a good idea to check whether value to write is valid.
 *It is also a good idea to do so atomically!
 */
void set_motor(/*parameters*/) {

    Serial.println("Change the motor speed \n");

    check_speed_limits();

}
```

.ino file

- Is the main() function with setup() and loop().

robocup_template | Arduino 1.8.1

File Edit Sketch Tools Help



```
robocup_template | Arduino 1.8.1
File Edit Sketch Tools Help

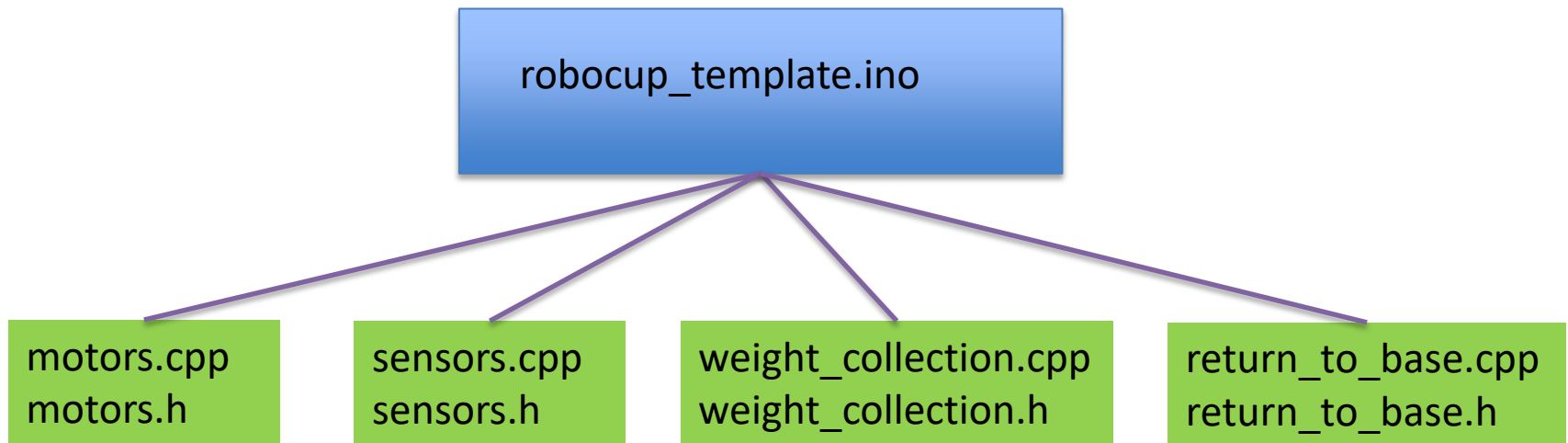
robocup_template | motors.cpp | motors.h | return_to_base.cpp | return_to_base.h | sensors.cpp | sensors.h | weight_co

/*****
 *
 *           ROBOCUP TEMPLATE
 *
 *
 * This is a template program design with modules for
 * different components of the robot, and a task scheduler
 * for controlling how frequently tasks should run
 *
 *
 * written by: Logan Chatfield, Ben Fortune, Lachlan McKenzie, Jake Campbell
 *
 *****/

#include <Servo.h>           //control the DC motors
//#include <Herkulex.h>      //smart servo
#include <Adafruit_TCS34725.h> //colour sensor
#include <Wire.h>            //for I2C and SPI
#include <TaskScheduler.h>   //scheduler
```

Template Structure

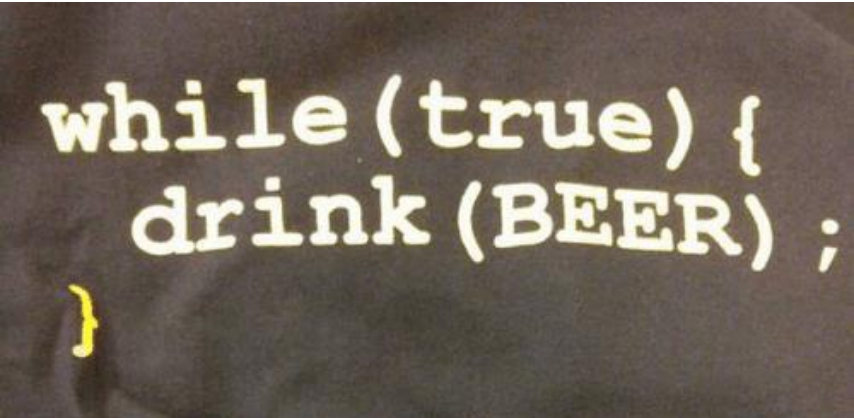
- Remember this is a template only and modify as you see fit



Task Management

- Your robot has a number of tasks to perform such as:
- Reading sensors, setting motors, picking up the weights, returning to base ...
- Many ways to perform these tasks including:
- Polling, interrupts, or use Arduino Scheduler

Infinite Loops



```
while(true) {  
    drink(BEER);  
}
```

- Infinite loop if no terminating condition or cannot be met
- For example, if you are waiting for a specific sensor to send data before you go to next task
- Your robot will just sit there doing nothing while the others pick up weights
- Happens every year!

Arduino Scheduler

- Execution of tasks (reading sensors, setting motors, picking up weights ...)
- Allows definition of:
 - Period for task (done every x ms)
 - Number of times task is executed
 - Priority of task (do most important first)

Scheduler Template Example

```
#include <TaskScheduler.h> //scheduler

#define US_READ_TASK_PERIOD 40

#define US_READ_TASK_NUM_EXECUTE -1

/* The first value is the period, second is how many times it executes
   (-1 means indefinitely), third one is the callback function */
Task tRead_ultrasonic(US_READ_TASK_PERIOD, US_READ_TASK_NUM_EXECUTE, &read_ultrasonic);

Scheduler taskManager; //create the Scheduler

void task_init() { //initialize and add tasks to the taskManager

    // This is a class/library function. Initialise the task scheduler
    taskManager.init();

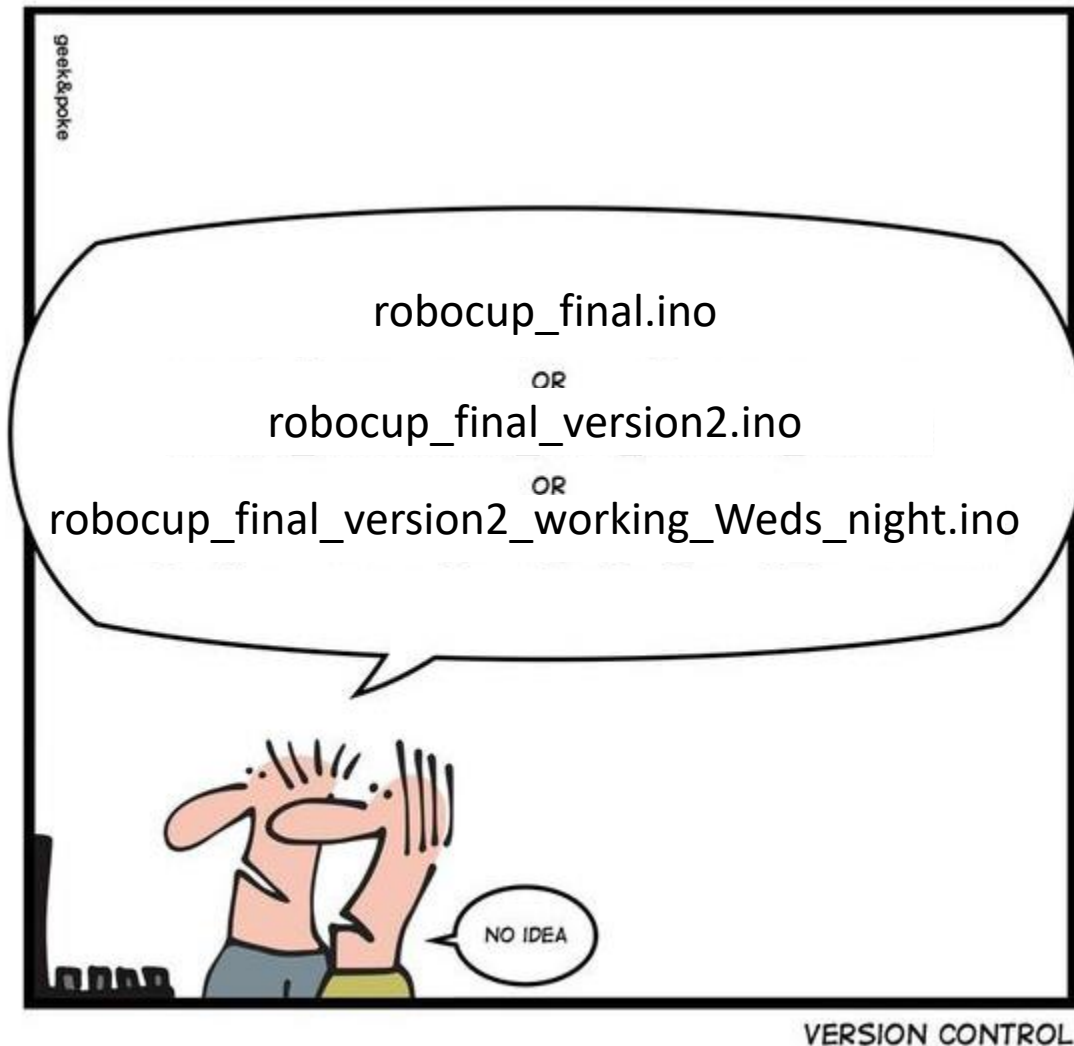
    // Add tasks to the scheduler
    taskManager.addTask(tRead_ultrasonic); //reading ultrasonic

    //enable the tasks
    tRead_ultrasonic.enable();
}

void setup() {
    task_init();
}

void loop() {
    taskManager.execute(); //execute the scheduler
}
```

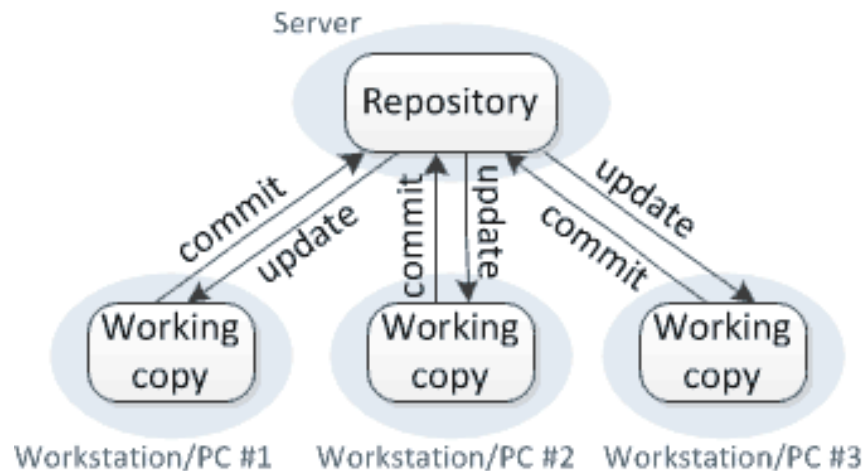
Why use version control?



Why use version control?

- Tracks changes
- Allows them to be easily undone
- Supports independent parallel development
- Essential for collaboration

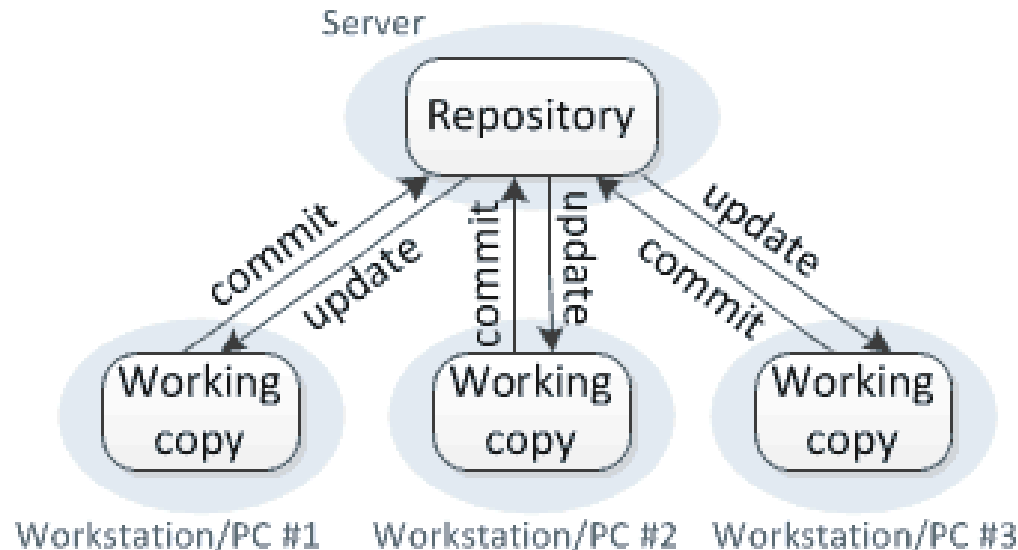
Centralized version control



Version Control Systems

- Many different systems exist:
- cvs (Concurrent Versioning System)
- svn (Apache Subversion)
- git

Centralized version control



git



- Started in 2005 by Linus Torvalds
- Most widely used software version control system today
- git is the system
- Many commercial repositories using git exist such as github, bitbucket



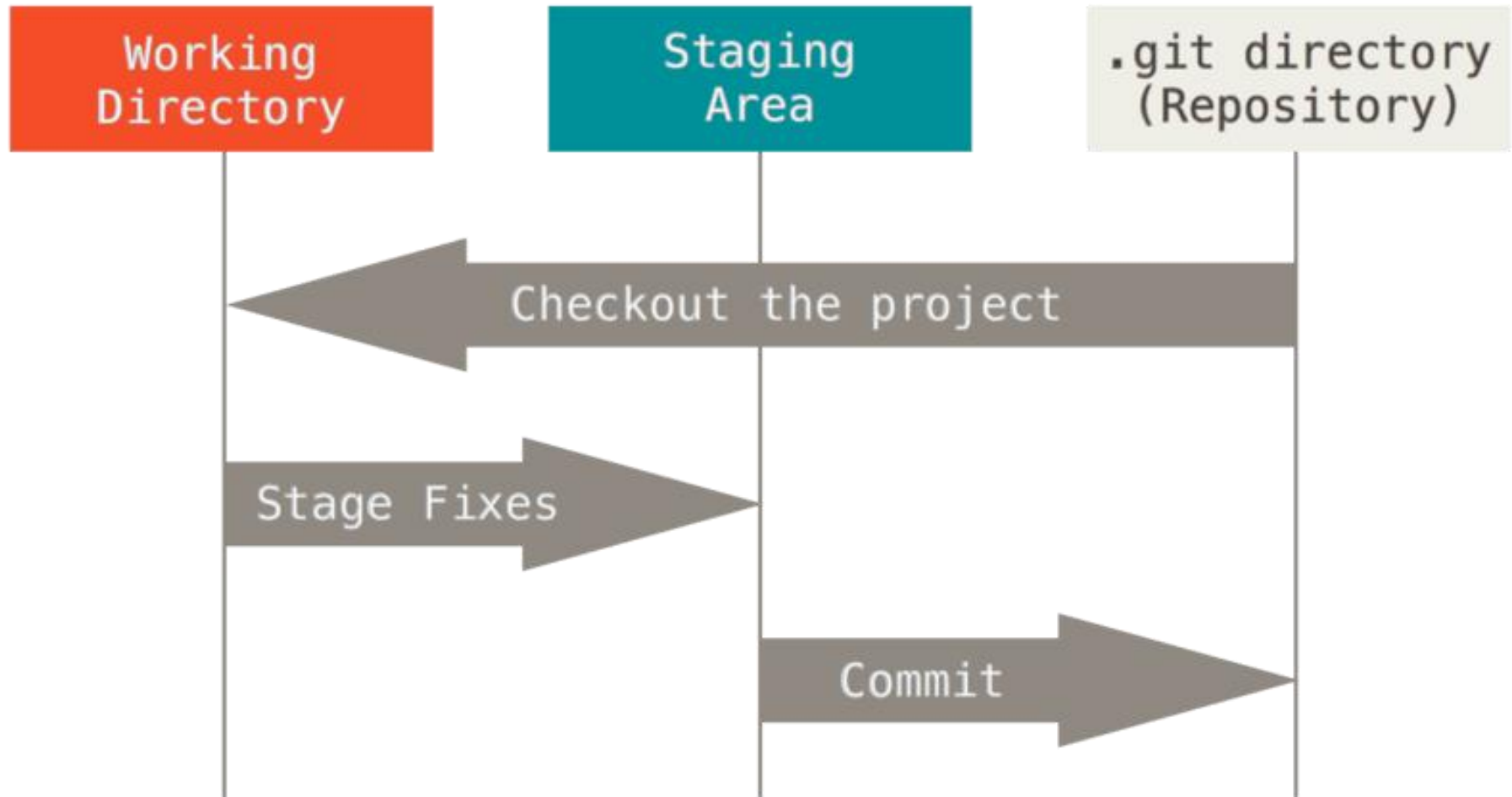
gitlab



GitLab

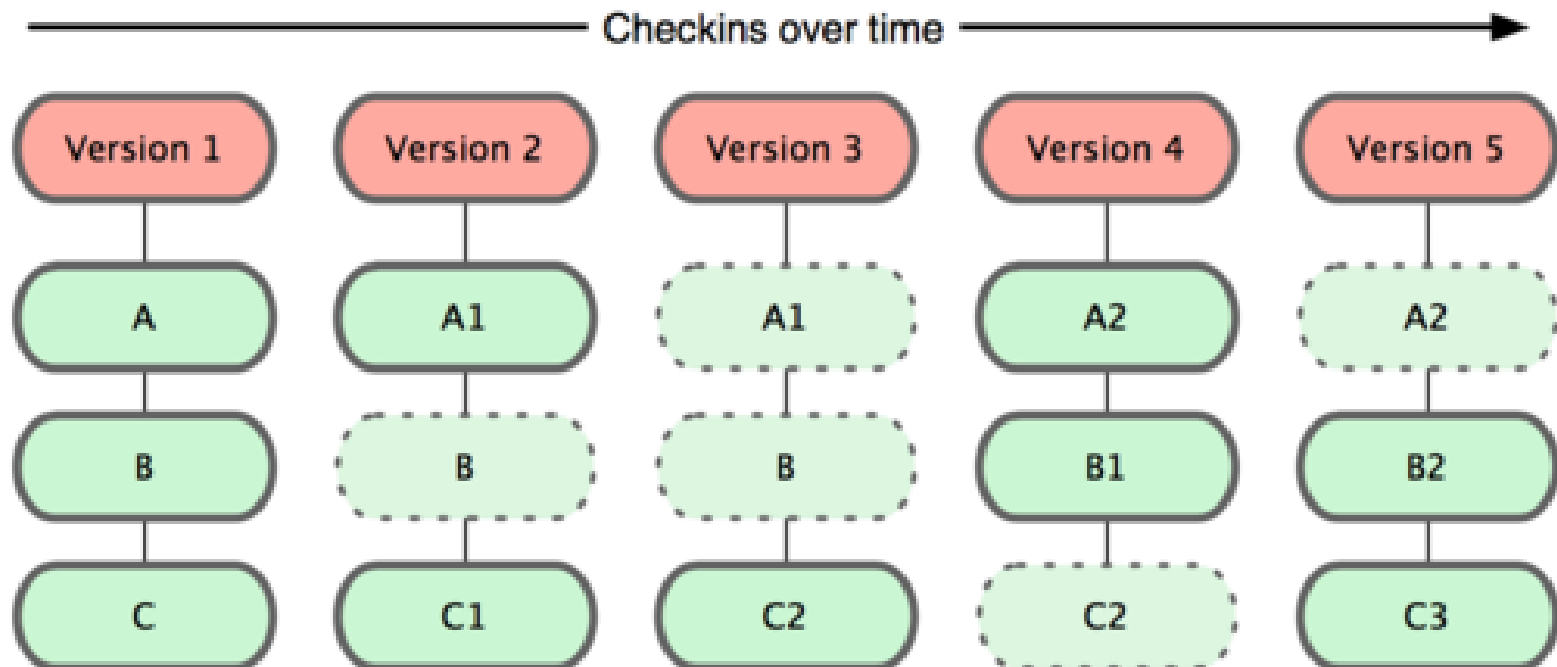
- AT UC, we can create free private web-based repositories with gitlab
- Uses your UC login and password
- <https://eng-git.canterbury.ac.nz>
- Need password or ssh key uploaded to server

How does git work?



How does git work?

- Takes a snapshot of your files each time you commit changes to the repository
- Only takes snapshot of files that have changed



Using git at the command line

- All git commands are of the form:
- git verb
- Right click in your P: (or documents) and open **Git Bash Here**

 MINGW64:/c:/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301

```
UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301
$ |
```

Creating a repository

- `mkdir robocup` (creates a directory)
- `cd robocup` (change to that dir)
- `git init` (makes this dir a repository)
- `git status` (gives status of repository)

Creating a repository

```
UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TE
$ mkdir robocup

UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TE
$ cd robocup

UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TE
$ git init
Initialized empty Git repository in c:/Users/rmc84/Dropbox/Lectu

UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TE
$ ls

UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TE
$ ls -a
./ ../ .git/

UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TE
$ git status
On branch master

Initial commit

nothing to commit (create/copy files and use "git add" to track)
```

Tracking changes

- Open an editor (Notepad ++) and enter a line of text. eg
- “mechatronics = mechanical + electronics”
- (put an enter/return at end of the line)
- Save as eg mechatronics.txt
- `git status` (give status of repository)

Tracking Changes

```
UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301/
$ vim mechatronics.txt

UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301/
$ git status
On branch master

Initial commit

Untracked files:
  (use "git add <file>..." to include in what will be committed)

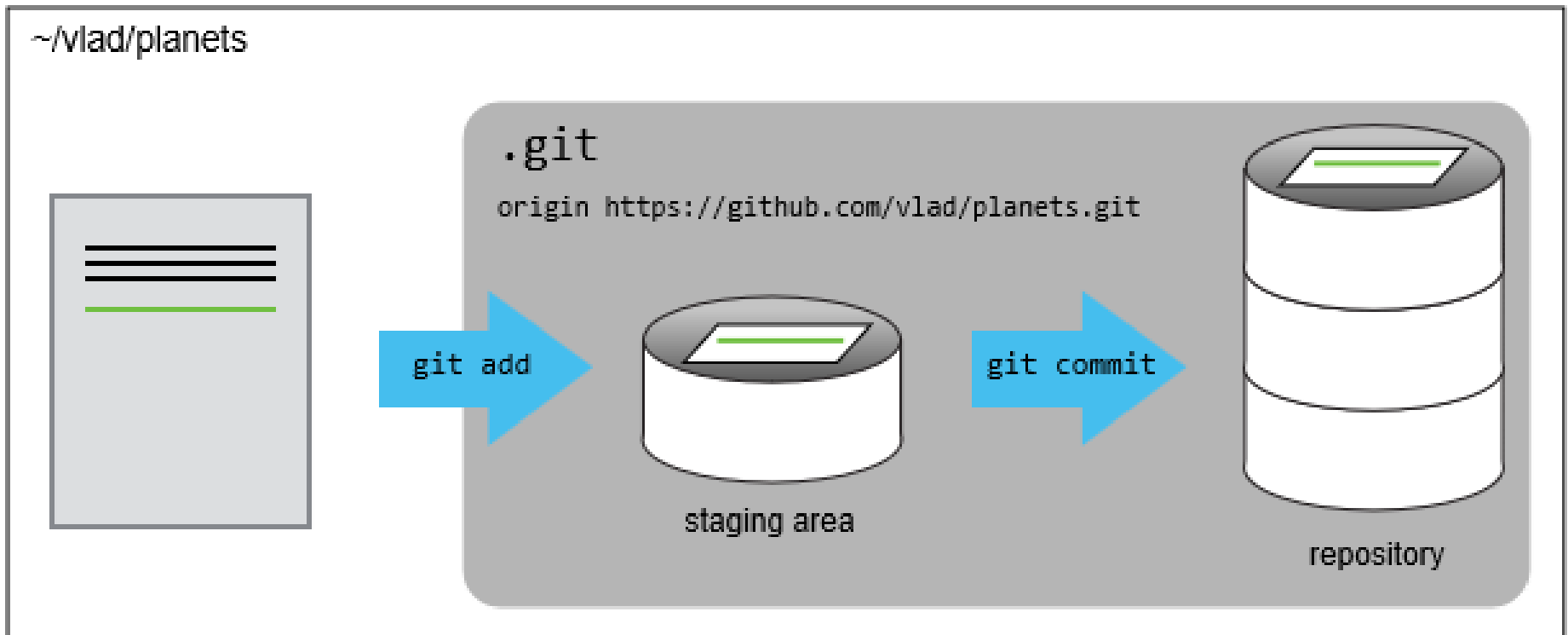
    mechatronics.txt

nothing added to commit but untracked files present (use "git add" to track)

UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301/
$ |
```

Committing files to a repository

- 2 step process. `git add` says we are keeping track of that file. `git commit` stores a copy in the local `.git` repository



Committing files

- `git add mechatronics.txt`

(tell the repo we are going to track this file)

- `git commit -m 'Defined mechatronics'`

(commit all changes in tracked files to the repo and label with this message)

- `git status`

(shows status of our repository)

- `git log`

(shows log of when & what commits)

Committing files

```
UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lectu
$ git add mechatronics.txt
```

```
UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lectu
$ git commit -m 'Defined mechatronics'
[master (root-commit) 527f491] Defined mechatronics
1 file changed, 1 insertion(+)
create mode 100644 mechatronics.txt
```

```
UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lectu
$ git status
On branch master
nothing to commit, working tree clean
```

```
UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lectu
$ git log
commit 527f4912e18b580b34de8631fbec78f8aa95fdb4
Author: Richard Clare <richard.clare@canterbury.ac.nz>
Date: Tue Mar 21 14:45:58 2017 +1300
```

Defined mechatronics

Some configurations

```
UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301/
$ git commit -m 'Defined Mechatronics'
[master (root-commit) a0863e7] Defined Mechatronics
Committer: Richard Clare <rmc84@canterbury.ac.nz>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly:

    git config --global user.name "Your Name"
    git config --global user.email you@example.com

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

1 file changed, 1 insertion(+)
create mode 100644 mechatronics.txt

UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301/
$ git config --global user.name "Richard Clare"

UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301/
$ git config --global user.email richard.clare@canterbury.ac.nz
```

Exploring History

- Make a change to mechatronics.txt then
- `git diff mechatronics.txt` (shows the difference to the file in the repository)
- `git diff HEAD~0 mechatronics.txt` (shows the difference to N commits previous in the repository)

Exploring History

MINGW64:/c/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301/robocup

```
UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lec
$ git diff
diff --git a/mechatronics.txt b/mechatronics.txt
index a1fc0ba..661fd40 100644
--- a/mechatronics.txt
+++ b/mechatronics.txt
@@ -1,2 @@
  mechatronics = mechanical + electronics
+It's all about the robots
```

```
UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lec
$ git diff HEAD~0 mechatronics.txt
diff --git a/mechatronics.txt b/mechatronics.txt
index a1fc0ba..661fd40 100644
--- a/mechatronics.txt
+++ b/mechatronics.txt
@@ -1,2 @@
  mechatronics = mechanical + electronics
+It's all about the robots
```

Checkout past versions

- `git status` (should show that you have made a change)
- `git checkout mechatronics.txt` (discards the changes in the Working directory. ie restores saved version)
- `git status` (should show nothing to commit again)

Checkout

MINGW64:/c/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301/robocup

```
UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301/robocup
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

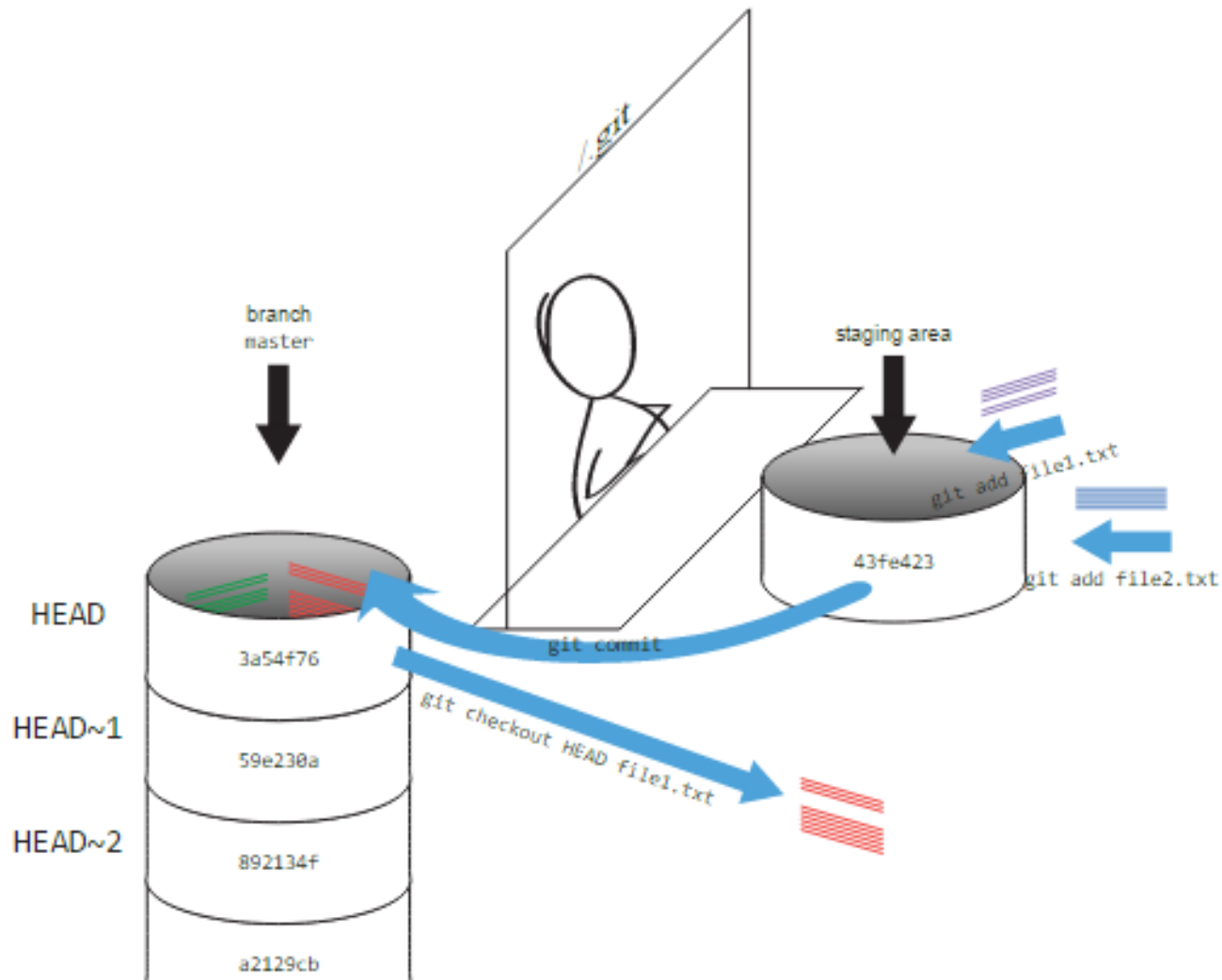
        modified:   mechatronics.txt

no changes added to commit (use "git add" and/or "git commit -a")

UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301/robocup
$ git checkout mechatronics.txt

UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301/robocup
$ git status
On branch master
nothing to commit, working tree clean
```

HEAD

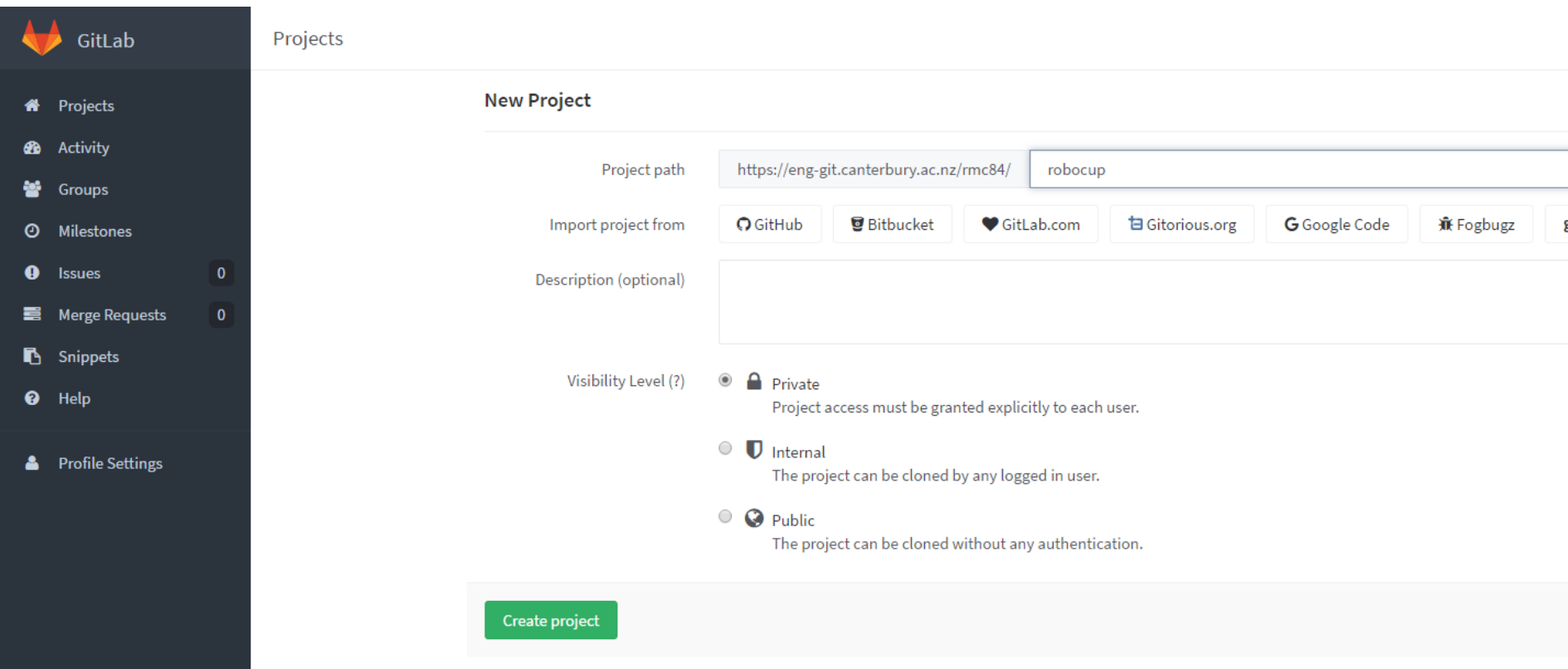


Checkout past versions

- `git checkout HEAD` (restores most recent version)
- `git checkout HEAD~1` (restores second most recent version)
- `git checkout HEAD~2` (restores third most recent version)

Pushing to remote repository

- Login to <https://eng-git.canterbury.ac.nz/> with your UC credentials



The image shows the 'New Project' form in the GitLab web interface. On the left is a dark sidebar with navigation links: Projects, Activity, Groups, Milestones, Issues (0), Merge Requests (0), Snippets, Help, and Profile Settings. The main content area is titled 'New Project' and contains the following fields:

- Project path:** A text input field containing 'https://eng-git.canterbury.ac.nz/rmc84/' and a dropdown menu showing 'robocup'.
- Import project from:** A row of buttons for GitHub, Bitbucket, GitLab.com, Gitorious.org, Google Code, and Fogbugz.
- Description (optional):** A large empty text area.
- Visibility Level (?):** Three radio button options:
 - ☒ **Private**: Project access must be granted explicitly to each user.
 - ☐ **Internal**: The project can be cloned by any logged in user.
 - ☐ **Public**: The project can be cloned without any authentication.

At the bottom of the form is a green button labeled 'Create project'.

Pushing to remote repository

- `git remote add origin`

`git@eng-git.canterbury.ac.nz:rmc84/robocup.git`

(copy and paste 2nd line from your gitlab window)

(this adds your dir to the remote repo)

- `git push origin master` (pushes changes from your local repository to the remote one)
- You should now be able to see your code on the gitlab website

Pushing to remote repo

MINGW64:/c/Users/rmc84/Dropbox/Lecturer/TEACHING/ENMT301/robocup

```
UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TEACHING/E
$ git remote add origin git@eng-git.canterbury.ac.nz:rmc84/robocup.git

UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Dropbox/Lecturer/TEACHING/E
$ git push origin master
Counting objects: 3, done.
Writing objects: 100% (3/3), 266 bytes | 0 bytes/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To eng-git.canterbury.ac.nz:rmc84/robocup.git
 * [new branch]      master -> master
```

- On gitlab you can then browse through all commits and changes

gitlab view commits

· Commits

Commits 2

Network

Compare

Branches 1

Tags 0

master

robocup

 21 Mar, 2017

2 commits

Updated def



Richard Clare authored less than a minute ago

Defined mechatronics



Richard Clare authored about an hour ago

Cloning a repository

- On gitlab on left pane choose **Members**, and then add your robocup partners
- For your teammates to get this repository:
- `git clone git@eng-git.canterbury.ac.nz:rmc84/robocup.git`
(copy and paste 2nd line from gitlab)

GitLab

Go to dashboard

Project

Activity

Files

Commits

Builds

0

Graphs

Milestones

Issues

0

Merge Requests

0

Members

Labels

Wiki

Settings

Richard Clare / robocup · Members

Search in this project

Add new user to project

Import members

Users with access to this project are listed below.

People

Project Access

Add users to project

robocup project members (1)

Richard Clare rmc84

It's you

Adam Mir
aam80

Aaron Ang
aan56

Aaron Huang
zhu28

Aaron Stockdill
aas75

Aaron Wilson
atw46

Abby Jameson
aja135

Abel Svoboda

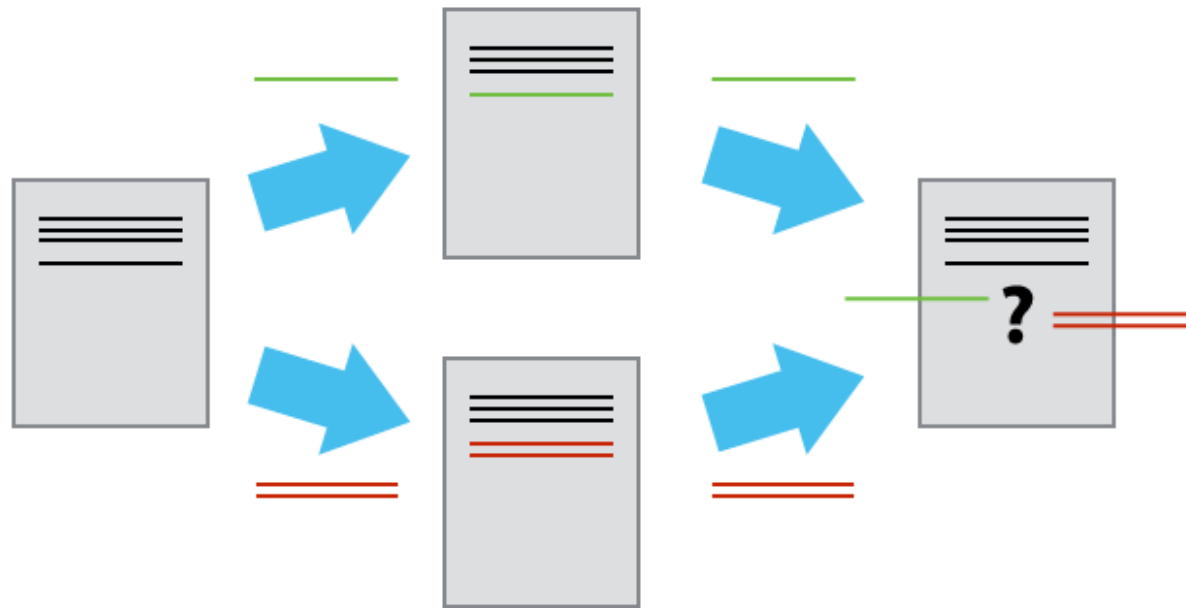
Collaborating

- We need to be up-to-date with our team before committing our changes
- Before you push your changes to the repository, you need to get your teammates' changes with:
- `git pull origin master`

```
UOCNT+rmc84@ELECST033 MINGW64 /c/Users/rmc84/Doc
$ git pull origin master
From eng-git.canterbury.ac.nz:rmc84/robocup
 * branch          master      -> FETCH_HEAD
Already up-to-date.
```

Conflicts

- What if you and your teammate edit the same line simultaneously?

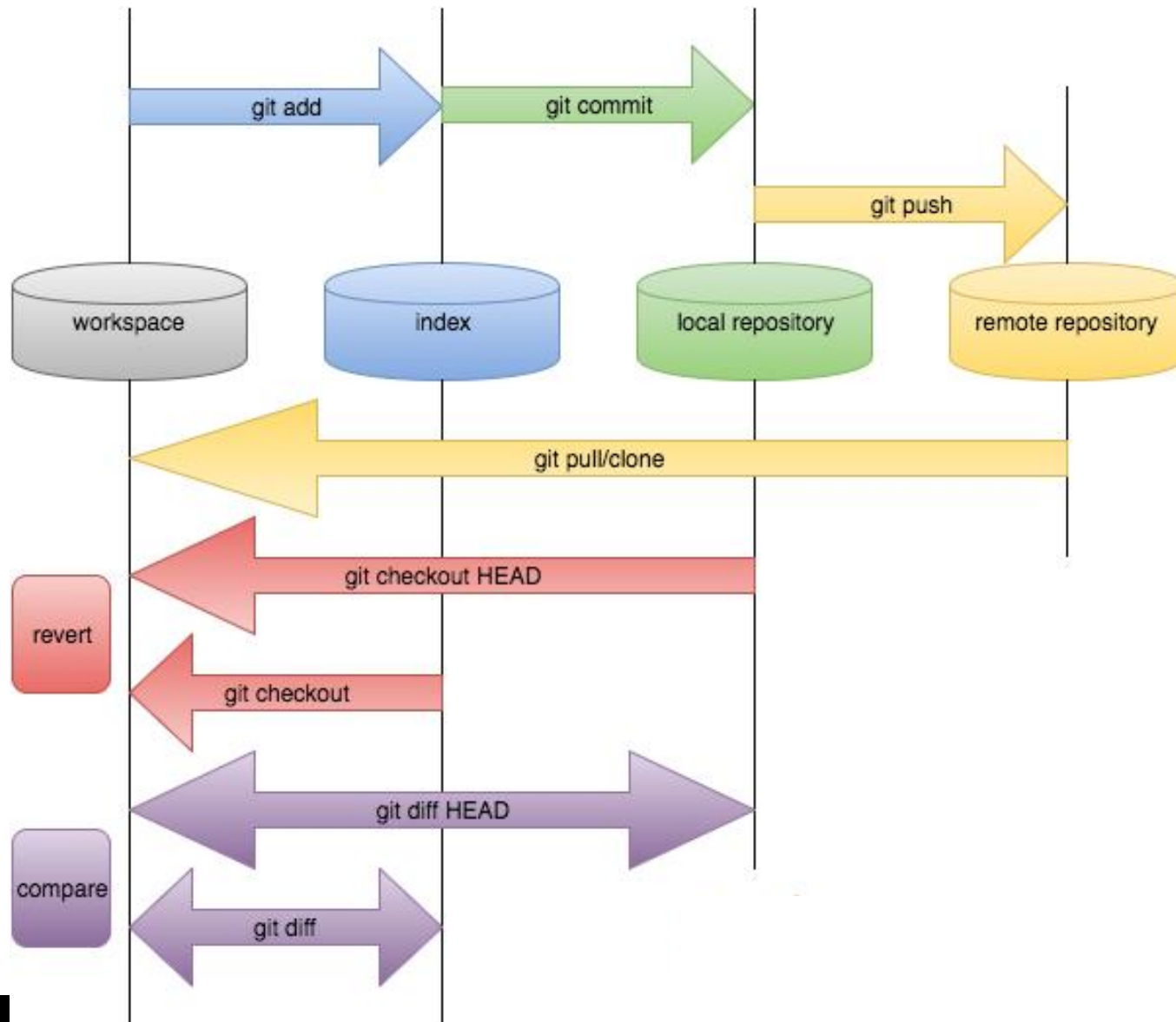


- Need to pull the changes from the remote, merge them into our local copy and push that!

Cheat sheet of useful commands

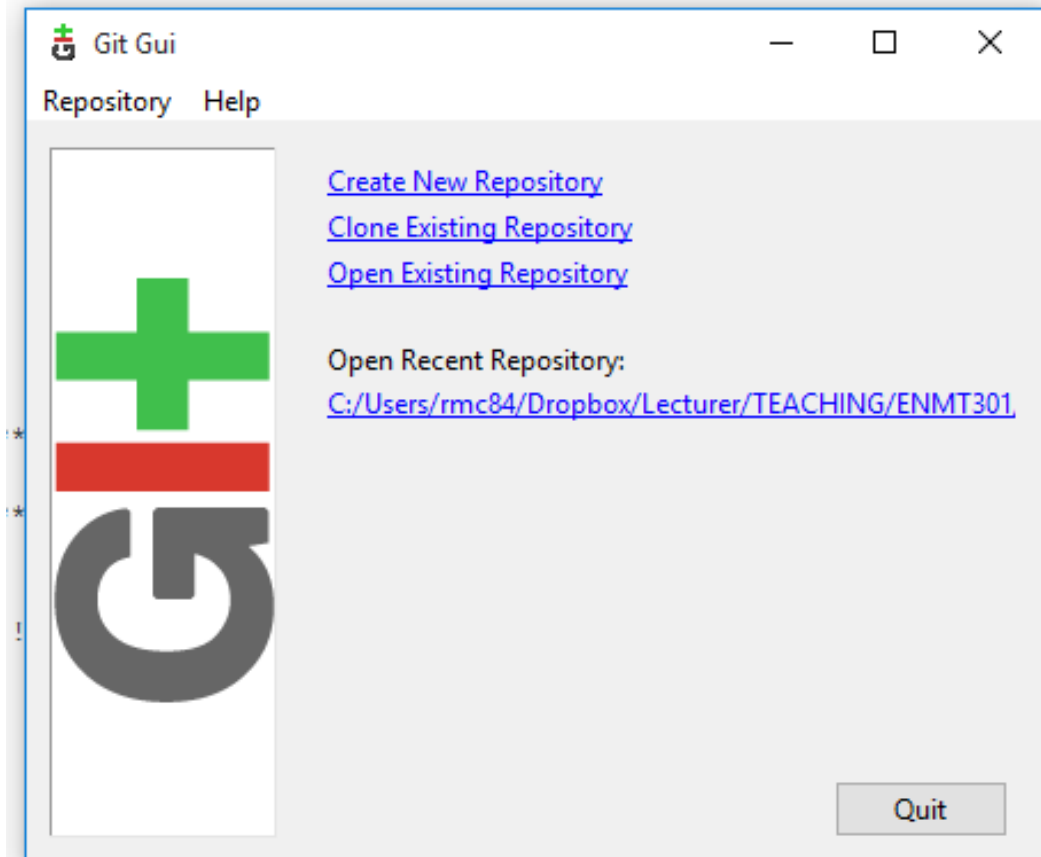
- `git init`
- `git add filename`
- `git commit -m 'Message'`
- `git push origin master`
- `git log`
- `git status`
- `git diff`
- `git pull`
- `git clone`
- `git remote add origin`
- `git blame`

A git summary

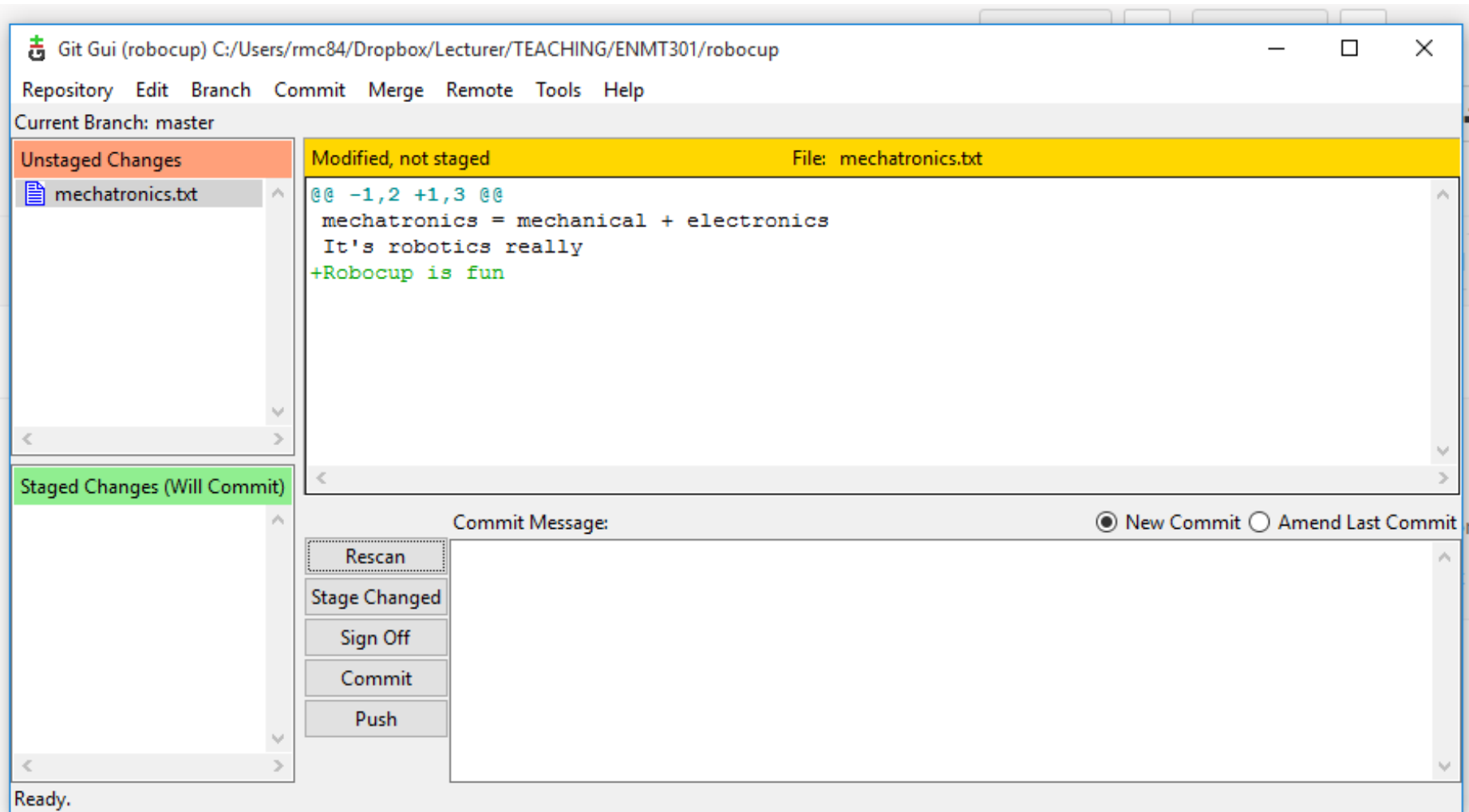


How to use Windows git GUI

- Create, Clone or Open a repository from the first Window



Git GUI-Same functionality but buttons



Your task now/later

- Copy the robocup template from LEARN to your git folder
- Add, commit and push these files to your remote repository
- The other teammates should then clone this repository to their drive
- You are now good to go with a version controlled software template for your robocup project!

Further reading

- This has been some (& not all) of the functionality of git
- For example, you may want to branch your code (stable, development etc)and work on different branches
- See these software carpentry notes for more...
- <http://swcarpentry.github.io/git-novice/>

The end!

- Be better than this!

